



MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE
REPUBLIC OF MOLDOVA

Technical University of Moldova Faculty of Computers, Informatics and
Microelectronics Department of Software and Automation Engineering

Lupasco Cristian FAF-233

Report

Laboratory work n.6

of Languages and Finite Automata

Checked by:

Cretu D., DISA, FCIM, UTM

Chişinău – 2024

Theory

Parsing is the process of taking a flat sequence of tokens (the output of lexical analysis) and discovering its hierarchical, grammatical structure. The result is often a parse tree (or concrete syntax tree), which mirrors the exact grammar rules used, including every punctuation and grouping.

In a compiler pipeline, parsing lets us verify that the input is well-formed, and it lays the groundwork for later phases (type checking, optimization, code generation).

A parse tree faithfully represents every production in the grammar, but it can be verbose and cluttered with nodes that exist only to reflect grammar details (e.g. “expression $\rightarrow$ term”). An Abstract Syntax Tree (AST), in contrast, strips away unnecessary intermediate nodes and focuses on the essence of each construct:

- Operators become interior nodes (+, *, etc.).

- Literals (numbers) and identifiers (variables) become leaves.

- Function calls collapse multiple tokens (sin, (, arg,)) into a single node.

Because an AST omits purely syntactic sugar, it’s easier to traverse for semantic analysis, optimizations, or code generation.

Conditions of the Problem

1. Get familiar with parsing, what it is and how it can be programmed;
2. Get familiar with the concept of AST;
3. Have a type TokenType (like an enum) that can be used in the lexical analysis to categorize the tokens;
4. Please use regular expressions to identify the type of the token;
5. Implement the necessary data structures for an AST that could be used for the text you have processed in the 3rd lab work;
6. Implement a simple parser program that could extract the syntactic information from the input text;

Technical Implementation

```
def lex(text: str) -> List[Token]:
    tokens: List[Token] = []
    for m in _master_pattern.finditer(text):
        tok_type = None
        for i, (_, ttype) in enumerate(_TOKEN_SPEC):
            if m.lastgroup == f'T{i}':
                tok_type = ttype
                break
        lexeme = m.group(0)
        if tok_type is None:
            continue # skip (whitespace)
        if tok_type == TokenType.INTEGER:
            value = int(lexeme)
        elif tok_type == TokenType.FLOAT:
            value = float(lexeme)
        else:
            value = lexeme
        tokens.append(Token(tok_type, value))
    tokens.append(Token(TokenType.EOF, None))
    return tokens
```

Purpose: The lex function takes an input string and uses a single compiled regular-expression to scan for token patterns defined in TOKENSPEC. For each match it determines the corresponding TokenType, converts numeric lexemes to int or float as needed, skips whitespace, and appends a Token(type, value) to the list. Once the entire string is processed it adds an EOF token and returns the complete token stream for parsing.

```

def factor(self) -> AST:
    """factor: (PLUS|MINUS) factor
    | INTEGER | FLOAT
    | IDENTIFIER
    | (SIN|COS) LPAREN expr RPAREN
    | LPAREN expr RPAREN
    """

```

Purpose: The factor method recognizes the smallest units in our grammar—unary operators, literals, variables, function calls, or parenthesized expressions—by looking at the current token’s type and “eating” it to advance the parser. If it sees a leading + or -, it constructs a UnaryOp node around a recursively parsed factor; for numbers and identifiers it builds Num or Var nodes. When encountering sin or cos, it consumes the function name and parentheses to create a FuncCall, and parentheses around an expression are handled by parsing the enclosed expr. Any unexpected token triggers an error.

```

def term(self) -> AST:
    """term: factor ((MULTIPLY|DIVIDE) factor)*"""
    node = self.factor()
    while self.current_token.type in (TokenType.
        MULTIPLY, TokenType.DIVIDE):
        op = self.current_token
        self.eat(op.type)
        node = BinOp(node, op, self.factor())
    return node

```

Purpose: The term method parses a sequence of factors connected by multiplication or division, ensuring those operators bind more tightly than addition and subtraction. It first parses a single factor, then in a loop checks for * or /; each time it finds one, it consumes the operator and the next factor, wrapping them in a BinOp node whose left child is the previously built subtree. When no more * or / tokens remain, it returns the resulting AST node representing the full term.

```
def expr(self) -> AST:
    """expr → term ((PLUS | MINUS) term)*"""
    node = self.term()
    while self.current_token.type in (TokenType.PLUS,
                                      TokenType.MINUS):
        op = self.current_token
        self.eat(op.type)
        node = BinOp(node, op, self.term())
    return node
```

Purpose: The `expr` method handles addition and subtraction by first parsing a term (which itself handles multiplication and division) and then looping as long as the next token is `+` or `-`. Each time it finds one of those operators, it consumes it, parses another term, and builds a `BinOp` node with the existing tree on the left and the new term on the right. When no more `+` or `-` tokens remain, it returns the complete AST for the expression.

Conclusions

In crafting this parser, I've developed a full pipeline that takes raw text through tokenization, syntactic analysis, and AST construction. I used a master regex to split input into TokenType-classified lexemes, then built a recursive-descent parser with factor, term, and expr methods to enforce operator precedence. The resulting AST node classes (Num, Var, UnaryOp, BinOp, FuncCall) capture the essential structure without extraneous grammar details. By cleanly separating lexing, parsing, and tree-building, I can now extend the language with new operators or functions, perform semantic checks, or generate code. Having this AST in place also sets the stage for evaluation, optimization passes, or compiling expressions into executable code. It also helps me visualize the structure and debug grammar rules more effectively.